Middlesex University
Computer Science Department

Dissertation/Project Coversheet

Student ID Number:	M	0	1	0	8	2	6	8	7
Student Name	Panchani Kevin								
Module Code:	CST4990								
Programme of Study:	MSc Computer Science								
Supervisor:	Elisabeth Fink								
Title:	AUTOMATED TECHNICAL DEBT DETECTION AND EVALUATION IN REACT-BASED WEB APPLICATIONS								
Declared Word Count:	8762								



AUTOMATED TECHNICAL DEBT DETECTION AND EVALUATION IN REACT-BASED WEB APPLICATIONS

By

Panchani Kevin

A DISSERTATION

Submitted to

MIDDLESEX University

Computer Science Department

in partial fulfillment of the requirements
for the degree of

MASTER OF SCIENCE

Computer Science

by Examination and Dissertation

Elisabeth Fink

September 2026

ABSTRACT

Technical debt in software engineering represents the implicit cost of rework due to the use of expedient solutions over architecture-aware ones. Modern frontend engineering features React-based web applications as a class of technical debt that cannot be automatically detected using static analysis systems designed to support generic languages. Tools like ESLint (OpenJS Foundation, 2013) and SonarQube (SonarSource, 2007) were meant to work with a wide array of languages and thus lack the semantic capabilities to detect React-specific maintainability issues such as large components, prop drilling, nested JSX elements, excessive use of stateful hooks, and high component coupling.

This dissertation designs, implements, and evaluates React Debt Analyzer – an automatic static analysis system based on AST that introduces the new React Component Debt Index (RCDI): a five-dimension weighted composite scoring measure designed specifically for React applications. Instead of using a penalty-subtraction scoring approach, RCDI implements a composite scoring mechanism based on SQALE concepts (Letouzey, 2012), theories of coupling and cohesion (Alzamil, 2023; Martin, 2002), and specific React code smell research (Ferreira and Valente, 2022). The five dimensions – Structural Complexity, State Complexity, Coupling Index, Prop Depth, and Component Cohesion – are combined as follows: RCDI = 0.25·SC + 0.20·StC + 0.20·CI + 0.20·PD + 0.15·CC.

Additionally, the system proposes the concept of the novel Rerender Propagation Score (RPS) – a measure without any precedence in technical debt literature that calculates the risk of a component rerendering as the product of state complexity and afferent coupling. The system takes as input a public GitHub URL, clones the repository, constructs an import coupling graph, parses all JS/JSX files with Babel AST, detects seven types of debt, and generates a five-tab interactive dashboard containing per-component RCDI analysis results, RPS risk ranking, and file treemaps. The evaluation performed with actual React repositories shows the system is capable of discriminating between codebases of different qualities and detecting debts not caught by ESLint.

DECLARATION

I hereby certify that this dissertation constitutes my own product, that where the language of others is set forth, quotation marks so indicate, and that appropriate credit is given where I have used the language, ideas, expressions or writings of another.

I confirm that I have not copied material from another source nor committed plagiarism nor commissioned all or part of the work (including unacceptable proof-reading) nor fabricated, falsified or embellished data when completing the attached piece of work.

I declare that the dissertation describes original work that has not previously been presented for the award of any other degree of any institution.

KEVIN PANCHANI

ACKNOWLEDGEMENTS

It is also my pleasure to extend my appreciation to my dissertation advisor, Elisabeth Fink, for her support and constructive suggestions during the course of this study. She suggested me to explore new ways of measuring technical debt related to React which became the foundation of the RCDI framework.

It is also my pleasure to thank all members of faculty and the staff of the Department of Computer Science at Middlesex University. It is also necessary to thank the open source communities that have created Babel, React, MongoDB, and Node.js.

Contents

ABSTRACT 3
ACKNOWLEDGEMENTS 5
Chapter 1: Introduction 8
1.1 Background and Motivation 8
1.2 Problem Statement 9
1.3 Aim and Objectives 10
1.4 Research Questions 11
1.5 Scope and Limitations 11
Chapter 2: Literature Review 12
2.1 Technical Debt — Concept and Origins 12
2.2 Static Analysis for Code Quality 12
2.3 Cyclomatic Complexity and Cognitive Load 13
2.4 Coupling, Cohesion, and the React Import Graph 13
2.5 React-Specific Code Smells and Debt Patterns 14
2.6 Existing Tools — ESLint and SonarQube 15
2.7 Summary and Research Gap 15
Chapter 3: System Design and Architecture 16
3.1 Architectural Overview 16
3.2 Backend Architecture 16
3.3 Frontend Architecture 17
3.4 Data Model 17
3.5 API Design 17
Chapter 4: Implementation — RCDI Framework, Detection Rules, and Scoring Engine 18
4.1 The React Component Debt Index (RCDI) — Theoretical Grounding 18
4.2 The Five RCDI Dimensions 18
4.2.1 Structural Complexity (SC) — weight 0.25 19
4.2.2 State Complexity (StC) — weight 0.20 20
4.2.3 Coupling Index (CI) — weight 0.20 20
4.2.4 Prop Depth Score (PD) — weight 0.20 21
4.2.5 Component Cohesion (CC) — weight 0.15 21
4.3 AST Parsing Engine 22
4.4 The Seven Detection Rules 22
4.5 Rerender Propagation Score (RPS) — Novel Metric 24
4.6 Scoring Model 24
Chapter 5: Technology Stack and Deployment 26
5.1 Technology Overview 26
5.2 Key Technology Justifications 26
Chapter 6: Evaluation 28

6.1 Methodology 28
6.2 Test Repositories and RCDI Results 28
6.3 Comparison with ESLint 31
6.4 Temporal Debt Analysis – Commit-History Timeline 31
6.5 Output 32
Chapter 7: Discussion 35
7.1 Interpretation of RCDI Results 35
7.2 Theoretical Contribution of RCDI 35
7.3 Limitations 36
7.4 Research Contribution Summary 36
Chapter 8: Conclusion and Future Work 37
8.1 Conclusion 37
8.2 Future Work 37
References 38
Appendix A: Tool Comparison Table 38
Appendix B: API Endpoint Reference 41
Analysis Resource 41
Reports Resource 41
Utility 41

Table of Figure

1 Dashboard……..32

2 Issues……..33

3 Score Breakdown……...33

4 History Timeline……..…34

Table

Table 1: RCDI Dimension……..…19

Table 2: scoring model……..24

Table 3: Technology Overview……...26

Table 4: full result……..…29

Table 5: Comparison table……..…40

Chapter 1: Introduction

1.1 Background and Motivation

React has grown into the most popular library for making user interfaces using JavaScript programming language. “At present, 2024, according to Stack Overflow, more than 40% of global web developers are using React (Stack Overflow, 2024)”. This has resulted in a significant problem which, to date, has not been properly addressed: the formation of React's component-specific technical debt. The notion of technical debt was introduced by Ward Cunningham (1992) to denote the future costs of rework resulting from choosing expedient solutions over architecturally-sound ones.

The debt can be manifested in a variety of forms in React applications different from those in traditionally structured codebases of the back-end type. For example, such manifestations include component overgrowth, excessive prop propagation, nested JSX, usage of too many hooks, afferent coupling of components. The problem cannot be detected with currently available general-purpose tools. Hence, the current research aims to develop an instrument which would allow measuring React-specific debt. Specifically, it involves developing React Component Debt Index (RCDI): an innovative theoretically-grounded composite scoring technique dedicated to React component model.

The motivation for development of the RCDI came from supervisor feedback regarding the initial penalty-subtraction scoring model lacking theoretical foundation and failing to take into consideration the React-specific structural features like component coupling, JSX-to-logic cohesion and the risks of component rerenders. These limitations became evident after systematic review of technical debt literature.

1.2 Problem Statement

Currently available static analysis tools are not able to properly detect and evaluate technical debt in web apps based on the React framework, since they do not have sufficient semantic specificity to properly analyze React's component structure, hooks, and JSX code. This problem is directly measurable and creates a deficiency in software quality assessment. The current set of general purpose tools does not provide React developers with an automated way to assess maintainability. More precisely:

- There is no available general purpose tool that analyzes JSX nesting depth as an indicator of complexity of components.
- There is no tool that analyzes prop drilling and evaluates its severity.
- There is no tool that analyzes usage of useState hook per each component.
- There is no tool that analyzes afferent coupling of React components.
- There is no tool that computes maintainability score based on coupling and cohesion concepts specific to React ecosystem.

1.3 Aim and Objectives

The main goal of this research is to develop a React-oriented technical debt detection tool using a unique composite scoring model called React Component Debt Index (RCDI) that will yield numerical, theory-based maintainability scores along with relevant problem reports in JavaScript/JSX code bases.

More specific goals are as follows:

- To perform a literature review of the technical debt detection approaches and find out the gaps in React-related technical debt detection in the literature published in the years 2020–2026.
- To design the RCDI model: five-dimensional composite weighted scoring system based on the SQALE methodology, coupling/cohesion theories, and research on React smells.
- To develop seven AST-based technical debt detection rules focusing on React-oriented technical debt patterns.
- To present a unique metric called Rerender Propagation Score (RPS) measuring the likelihood of a component re-rendering due to state complexity and afferent coupling.
- To construct a full stack web application for interactive technical debt detection without local tools installation.
- To assess the tool's performance using real React codebases in comparison with ESLint.

1.4 Research Questions

This dissertation addresses the following research questions:

1. How well do current static analysis techniques detect maintainability-related technical debt in React applications?
2. Which frontend-specific maintainability problems are not detected by general technical debt tools?
3. Is there an effective way to use static analysis techniques to detect technical debt in React based applications?

1.5 Scope and Limitations

In this study, the analysis shall be performed on those GitHub public repositories which contain .js and .jsx code files only. Languages such as TypeScript, Python, Java and other programming languages do not apply to this case. This framework does not offer any profiling, security analysis and dependency vulnerability analysis. Using the value of afferent coupling of files as an approximation of component coupling, the Coupling Index dimension will be calculated. Analysis will only be performed on React public repositories.

Chapter 2: Literature Review

2.1 Technical Debt — Concept and Origins

The notion of technical debt was first coined by Ward Cunningham (1992), who used it as an analogy of taking money on loan with the intention of sacrificing code quality in favor of immediate delivery. This work was further developed by Kruchten et al. (2012) by differentiating between deliberate and non-deliberate debts and by dividing them into design, code, test, documentation, and infrastructure categories. Alves et al. (2016) performed systematic mapping and found that technical debt literature is overwhelmingly concentrated on Java and C++ backends with a negligible number of papers on JavaScript or front end systems – thus, explaining the relevance of this research.

In a recent systematic mapping of the technical debt quantification methods, Perera et al. (2024) have reviewed 89 primary studies and revealed that almost all scoring models have no empirical calibration and disregard specific structural properties of the programming language.

2.2 Static Analysis for Code Quality

The static analysis of the program, i.e., the analysis of the source code without executing it, is the main tool for automated detection of software quality. Aho et al. (2007) explain how to build abstract syntax trees (ASTs), as a basis for semantic analysis, thus laying the theoretical grounds for Babel-based parsing engine, utilized in this research. According to Livieri et al. (2007), AST-based approach is much more effective than text-based one in terms of structural pattern detection, which makes @babel/parser preferable to the alternative tokenizers.

Campbell and Papapetrou (2013) introduce the concept of SQALE approach, according to which the technical debt is calculated as the sum of remediation costs measured along the number of different quality dimensions considered separately. While RCDI uses the multi-dimensional approach suggested by SQALE, in order to increase the readability, RCDI replaces the measure of remediation time with the normalized 0-100 scale.

2.3 Cyclomatic Complexity and Cognitive Load

McCabe (1976) introduced cyclomatic complexity as the numeric criterion for counting the number of linearly independent paths through the source code. Guidelines based on empirical evidence by Watson and McCabe (1996) have set the threshold values, such that functions having complexity above 10 cannot be properly tested, while those exceeding 20 are considered highly risky. Such values are used explicitly to assign severity levels for detecting cyclomatic complexity.

Gill and Kemerer (1991) provided empirical evidence about the relation between high cyclomatic complexity and high maintenance costs. This empirical proof supports the use of complexity as the highest weighted dimension ($\alpha = 0.25$) in RCDI metric formula. JSX depth multiplier of 1.5 in Structural Complexity dimension is based on cognitive load research. Abbes et al. (2011) stated that each additional nesting level of code increases the amount of context frames in working memory by one, thus increasing its cognitive load in comparison with logical nesting. Cognitive load research by Miller (1956) about limits of working memory capacity of seven items ±2 explains the State Complexity threshold, at which more than ten useState hooks mean that the capacity of human mind is reached.

2.4 Coupling, Cohesion, and the React Import Graph

Coupling and cohesion are fundamental qualities of the software product. According to Robert Martin (2002), the fan-in (afferent coupling) is defined as the number of modules that depend on a particular module. The higher the fan-in, the greater the fragility, because changes in widely imported components put at risk all of their dependents. The instability measure $I = Ce / (Ca + Ce)$ (Ca is afferent coupling, Ce is efferent coupling) defined by Robert Martin serves as a theoretical background for the Coupling Index in RCDI.

Alzamil (2023) proved the connection between the level of coupling and the low code quality empirically, by analysing software components in large-scale research and discovering statistically significant associations between afferent coupling and maintenance effort. Iftikhar et al. (2024) analysed 165 primary studies connecting the source code metrics to

external software quality attributes and found that the coupling metrics predict fault-proneness. These discoveries make the Coupling Index a vital part of RCDI.

Component Cohesion in RCDI is based on the single responsibility principle (Martin, 2002) that states that every component must have only one responsibility. JSX-to-logic ratio (JSX lines / total component lines) is an implementation of this principle – the component in which JSX takes approximately half of the component size is well-balanced.

2.5 React-Specific Code Smells and Debt Patterns

A large body of research on React specific smells was done by Ferreira and Valente (2022) and they identified and characterized four such smells namely: Large Component, Prop Drilling, Unnecessary Re-render and Direct DOM Manipulation, which were found to be the most common and significant. Their analysis of 508 open-source React repositories established the correlation between those patterns and poor maintainability and high issue density. The basis of all seven detection rules of the current system lies in the aforementioned classification.

Nunes et al. (2024) took this study further, exploring TypeScript React applications and discovering that such patterns as hook misusage (excessive usage of useState and lack of useCallback memoisation) are among the most common smells in typed React repositories. The results obtained by the authors support both State Complexity and RPS dimensions of RCDI.

Liu et al. (2021) proposed DebtHunter, a machine learning algorithm to detect self-admitted technical debt based on comments in the code. Although the approach was shown to be highly accurate, it does not deal with structural smells and hook usage in JSX, thus confirming the complementarity of the AST-based approach used in the current research.

Verdecchia et al. (2022) have offered ATDx, an architectural technical debt index evaluated for Apache and ONAP ecosystems. ATDx is architectural in nature and does not go down to the component level.

2.6 Existing Tools — ESLint and SonarQube

ESLint (OpenJS Foundation, 2013) is a widely-used linting utility for JavaScript programming. It utilizes the concept of AST and implements numerous rules which are configurable, yet in those that are related to React (eslint-plugin-react), only the absence of some important props, wrong hooks order, and similar syntactic details are checked. ESLint is unaware of component coupling, JSX nesting level as a measure of code quality, or a number of hooks as an indicator of bad state management (Zaef, 2021). SonarQube (SonarSource, 2007) offers maintainability ratings but analyzes code in more than 30 languages at once, hence is forced to limit itself in any particular environment. Its JavaScript analysis is limited to cyclomatic complexity and code duplication at file level.

2.7 Summary and Research Gap

AST-based analysis, measures of coupling/cohesion, and cyclomatic complexity are well-grounded as maintainability predictors from both theoretical and empirical perspectives based on the current literature. Yet, there is one evident gap that needs to be mentioned: currently, no existing tool integrates React-related semantics with multi-dimensional composite scoring based on the principles of coupling, SQALE, and research on React-related code smells. This dissertation fills in this gap via RCDI, which is more targeted compared to ESLint (React-related composite scoring).

Chapter 3: System Design and Architecture

3.1 Architectural Overview

React Debt Analyzer Architecture is composed of three layers: React 18 front-end, Node.js Express back-end, and MongoDB Atlas database service in the cloud. The analysis takes place asynchronously. As soon as the user provides GitHub link to the analyzer, the back-end creates the record in the database and provides the unique identifier for analysis. The analysis itself consists of stages such as cloning, import graph generation, file scanning, AST parsing, rule execution, RCDI calculation and RPS calculation and is run in the background. Front-end queries a status of an analysis every three seconds, thus preventing HTTP time-out for large repositories.

An important architectural change introduced in the v2 version of the application is an additional import graph generation stage (Stage 4 of 8). It scans all the import statements in the entire repository and generates a dependency graph, calculating afferent coupling (fan-in) for each component file. Fan-in values are injected into per-component metrics before the RCDI calculation is carried out. As a result, Coupling Index and RPS dimensions become aware of the actual cross-file dependencies.

3.2 Backend Architecture

The backend is designed in a way that makes separation of concerns possible in eight folders:

- controllers/ - handling http requests and responses
- routes/ - Express routes (analysis, reporting resources)
- services/ - orchestration code (RCDI eight-step pipeline, Git functions)
- analyzers/ - RCDI engine, Import graph creator, Component Metrics Extractor, File Analyser
- rules/ - seven separate debt detection algorithms integrated into ruleEngine.js
- parsers/ - Babel ast parser configuration (astParser.js)

- models/ - Mongoose Schema with RCDI dimensions, RPS ranking, and component models
- middleware/ - cross-cutting error handling

3.3 Frontend Architecture

The front-end consists of a five-tabs dashboard consisting of Overview, RCDI Breakdown, Rerender Risk, Issues, and Files tabs. Some of the new charts in version 2 include the RCDIRadar (5-dimensional radar chart), RPSChart (bar chart of highest risk components), and ComponentRCDITable (individual component table for all 5 dimensions, can be sorted by RCDI, RPS, fan-in or hook count). All charts are wrapped in an ErrorBoundary component to avoid crashes in case of any rendering problems.

3.4 Data Model

There have been substantial changes to the MongoDB Analysis collection in version 2. The structure of each document includes: project-level RCDI score (0–100), letter grade and label, five dimension scores (SC, StC, CI, PD, CC) in form of LOC weighted average values, an array of rpsRanking that contains the top ten components with the highest rerender risk and file analysis results that include an array of components in which all the fields include: name, filePath, loc, maxCyclomaticComplexity, maxJsxDepth, hookCount, fanIn, isMemoised, spreadPropCount, propChainDepth, jsxLines, totalLines, rcdi, rpsScore, rpsRaw and dimensions (all five scores).

3.5 API Design

There are ten REST API endpoints in the application. In addition to the initial eight, v2 introduces GET /api/reports/:id/rcdi (full report on RCDI with component details) and GET /api/reports/:id/rps (RPS rank). The response from the /rcdi endpoint is projectRcdi, grade, gradeLabel, dimensionScores, categoryScores, rpsRanking, and components list in descending order by the RCDI score. This makes it possible for the RCDI Breakdown tab to be displayed without an extra request.

Chapter 4: Implementation — RCDI Framework, Detection Rules, and Scoring Engine

4.1 The React Component Debt Index (RCDI) — Theoretical Grounding

RCDI represents a five-dimension weighted composite score taking the place of the v1 penalty-subtraction formula. The score is built on three lines of literature:

- SQALE (Letouzey, 2009; 2012) — debt defined as the aggregated quality distance, measured through independently calculated dimensions, all of which are normalised to 0-100 scale.
- Theories of coupling and cohesion (Alzamil, 2023; Martin, 2002; Iftikhar et al., 2024) — afferent coupling and cohesion ratio as the major predictors of software maintainability.
- React code smells (Ferreira and Valente, 2022; Nunes et al., 2024) — an empirical approach to identifying the structural pattern recognised by practitioners as debt in React projects.

RCDI differs from SQALE in representing debt as a normalised 0-100 score, which is easier to understand for the researchers and developers who lack knowledge about project-specific calibration factors. The composite formula is:

$$\mathbf{RCDI(component) = \alpha_1 \cdot SC + \alpha_2 \cdot StC + \alpha_3 \cdot CI + \alpha_4 \cdot PD + \alpha_5 \cdot CC}$$
$$\mathbf{where\ \alpha_1=0.25,\ \alpha_2=0.20,\ \alpha_3=0.20,\ \alpha_4=0.20,\ \alpha_5=0.15,\ \Sigma\alpha_i = 1.0}$$

The RCDI at the level of the individual project is the weighted sum of all the components, as per SQALE's aggregation concept:

$$\mathbf{RCDI(project) = \Sigma(RCDI(c_i) \times LOC(c_i)) / \Sigma\ LOC(c_i)}$$

4.2 The Five RCDI Dimensions

The RCDI model assesses React-specific technical debt through five interrelated categories, where each reflects an independent aspect of component maintainability and potential architectural risks. The below sub-sections describe the five categories, how each is calculated, and justify the choice of weightage and scoring cut-off criteria.

Dimension	Symbol	Weight (α)	What It Measures
Structural Complexity	SC	0.25	Maximum cyclomatic complexity + JSX nesting depth × 1.5 per component
State Complexity	StC	0.20	Usage frequency of useState / useReducer / useRef hooks per component
Coupling Index	CI	0.20	Afferent coupling - the number of other files that import this component (fan-in)
Prop Depth Score	PD	0.20	Prop chain nesting level × 15 + spread-prop usage × 8 (penalty system)
Component Cohesion	CC	0.15	|JSX line count / total lines − 0.5| deviation from ideal balance ratio

Table 1: RCDI Dimension

4.2.1 Structural Complexity (SC) — weight 0.25

React structural complexity means that as a project grows, the codebase tends to get tighter coupling, maintenance costs, and cognitive friction This is because React is an opinion-less UI library and not a framework that insists on a definite architecture. This makes it easy for developers to fall into structural problems.

SC is the most highly weighted dimension as cyclomatic complexity is the most validated predictor of fault proneness across all literature on software engineering (Gill and Kemerer, 1991; Watson and McCabe, 1996). For the React project, JSX nesting level is additionally included as an equally important variable – cognitive load studies show that for every nesting level there will be one additional context frame in working memory (Abbes et al., 2011), therefore deep JSX is proportionately heavier than the same level of branching.

$$SC_raw = \max(CyclomaticComplexity, JSX_depth \times 1.5)$$
$$SC_score = \max(0, 100 - \max(0, SC_raw - 10) \times 8)$$

The threshold is taken straight from Watson and McCabe (1996). The multiplier for JSX depth is based on cognitive load proportionality. For example, if a particular part has CC value = 8 and JSX nesting value = 7 then SC_raw = max(8, 10.5) = 10.5 and thus SC_score = max(0, 100 – 4) = 96.

4.2.2 State Complexity (StC) — weight 0.20

The StC metric measures stateful hook calls (useState, useReducer, useRef for mutable state) per component. The threshold is based on Miller's (1956) cognitive limits on working memory: a person can hold roughly seven things in working memory; when ten distinct state variables are involved, the developer needs to juggle ten state machines simultaneously – more than working memory can hold.

Score = Max (0, 100 - Min (1, hook_count/10) * 100)

A component with zero hooks will be scored as 100. A component with five hooks will receive a score of 50, while a component with 10 hooks will have a score of 0.

The implication of this score is that a component which is depended upon by thirty components cannot be edited independently. Editing of the props API or any exported variables in such a component will have to take into account all thirty components. This is exactly the maintenance challenge that afferent coupling encapsulates numerically. In React applications, highly-coupled components tend to be reusable UI abstractions - things like Buttons, Cards, or common layouts - which started out small and narrow in scope but grew over time.

4.2.3 Coupling Index (CI) — weight 0.20

Coupling Index (CI) is a formal metric for assessing interdependency of code modules in software engineering and React programming.

Although formal indices such as the Microservice Coupling Index (MCI) utilize mathematical calculations to measure relative dependencies in backend development, the front-end developer uses the idea of the Coupling Index as an indicator of structure strength. This index is used to determine the level of interdependence of React components, hooks, and modules.

A high Coupling Index means that changes in one component create a domino effect of problems and upgrades throughout your UI.

Fan-in indicates afferent coupling, that is, the number of other files in the repository that import this component. It is fan-in in Martin (2002) terminology, but now it is applied to the import graph in React. Large fan-in makes the component a fragility hotspot, since any

modification of it increases the fragility of its dependents. Fan-ins are calculated in the importGraphBuilder module that analyzes all static imports in the repository, resolving relative file paths to create a directed graph of dependencies.

$$\mathbf{CI_score = max\ (0,\ 100 - max\ (0,\ fan_in - 5) \times 10)}$$

The threshold value of 5 (components imported by 5 or fewer locations are considered not to be fragility issues) is derived from Martin's (2002) instability research. Each import above 5 deducts 10 points, so a component with a fan-in of 15 receives max (0, 100 - 100) = 0 points. There is no such aspect at all in the v1 and in all other generic tools analyzed.

4.2.4 Prop Depth Score (PD) — weight 0.20

The Prop Depth Score (PD) is a structural complexity metric for React architecture analysis that estimates the level of fragility in prop drilling in a component structure.

It represents the number of structural levels the state or data has to pass in order to reach the leaf component that utilizes it.

The Prop Depth Score indicates a very fragile UI structure. It shows that intermediate components have to be used as pipes through which the data flows.

PD measures prop drilling using prop chain depth and spread-prop number. They both are determined using AST analysis on JSXOpeningElement nodes. Prop drilling is named the most frequent React code smell by Ferreira and Valente (2022).

$$\mathbf{PD_score = max\ (0,\ 100 - chain_depth \times 15 - spread_count \times 8)}$$

In case of chain depth of 3 without spread props, PD = max (0, 100 – 45) = 55. In case of chain depth of 4 with 2 spread prop usages, PD = max (0, 100 – 60 – 16) = 24. Chain depth is now estimated as spread prop usages; cross-file tracing is mentioned as a future work direction.

4.2.5 Component Cohesion (CC) — weight 0.15

Component Cohesion (CC) refers to how related and cohesive the functionalities of a single component are in React.Inspired by the traditional software engineering concept of High Cohesion, Low Coupling, high Component Cohesion implies that a component does just one thing very well.Every line of code written in it, regardless of whether it concerns state

management, hooks, or markup, contributes to achieving its sole purpose. Low Component Cohesion results in bloated “God Components.”

CC evaluates the implementation of the principle of one responsibility based on the ratio of JSX rendering code to the number of total lines of code for components (Martin, 2002). An ideal ratio of 50% means that the component implements an appropriate balance of logic and rendering operations. Too high and too low ratios imply extreme situations when a component is either a pure template or a pure logic module.

$$\mathbf{CC_score = max\ (0,\ 100 - |jsx_ratio - 0.5| \times 200)}$$

For ratio = 0.5 (ideal): CC = 100

For ratio = 0.9 (template only): CC = max (0, 100 - 80) = 20

For ratio = 0.1 (logic only): CC = 20

There is no similar dimension in any existing tool.

4.3 AST Parsing Engine

Parsing of all code is based on @babel/parser with a full set of plugins such as jsx, classProperties, classPrivateProperties, decorators-legacy, optionalChaining, nullishCoalescingOperator, dynamicImport, object. This setup guarantees compatibility with almost any modern React codebase. The @babel/traverse module performs tree traversal, invoking the respective callback functions on particular nodes. Source code files, which cannot be parsed because of syntax errors in the source code, will get a parseError property, as well as an RCDI value of 100.

4.4 The Seven Detection Rules

Seven AST-based rules generate the list of issues presented on the dashboard. Every rule is based on the React code smells taxonomy proposed by Ferreira and Valente (2022). Severity levels define how much certain dimensions of the RCDI are impacted, although now RCDI is the main score, not a penalty count.

- Rule 1 - Over-sized component: PascalCase functions with 150 / 300 / 500 lines get medium / high / critical severity. Based on the Large Component smell (Ferreira and Valente, 2022).

- Rule 2 - Prop Drilling: JSXSpreadAttribute ({...props}) and JSXOpeningElement nodes with 8+ properties are highlighted. Based on the Prop Drilling smell by Ferreira and Valente (2022).

- Rule 3 - Deep Nesting: JSX root trees with a depth of ≥ 4 / 5 / 6 get medium / high / critical severity. Based on the results of cognitive load research by Abbes et al. (2011).

- Rule 4 - Cyclomatic Complexity: Any function having CC greater than 10/15/20 gets medium/high/critical classification. On the basis of studies conducted by Watson and McCabe in 1996.

- Rule 5 - Long Function: Any function having more than 40/60/100 lines becomes medium/high/critical in severity.

- Rule 6 - Duplicate Code: Intra-file sliding-window normalisation detects duplicated 6 lines of code. Based on the approach by Livieri et al. (2007).

- Rule 7 - Excessive useState: Components with 5 / 7 / 10+ hooks get medium / high / critical. Based on the cognitive load by Miller (1956) and Ferreira and Valente (2022).

Consequences of all these seven debt patterns for the long-term well-being of codebase are compound and profound. Components with high line counts and complex functions have an increased surface of defects - a bug in a 500-line component responsible for several features may be related to some interaction between these concerns and locating its cause takes quite a lot of time. Prop drilling and use of too many useState hook increase state management complexity, which gets bigger with each new feature; adding just one extra data requirement to the prop drilled architecture implies changing each component in the sequence, transforming small changes into big refactorings involving multiple files. Cyclomatic complexity of codebase can be used to estimate its fault proneness (Watson and

McCabe, 1996); a function with 20 branches has 20 different ways of being executed, which implies having 20 different test cases. Code duplication implies fixing a defect in multiple places at once; a code element that is both large in size, highly nested, and prop-drilled performs poorly on many aspects of RCDI and emerges as a candidate for refactoring.

4.5 Rerender Propagation Score (RPS) — Novel Metric

RPS is a metric that does not have any predecessors in the technical debt literature. RPS is used to fill in a gap pointed out by Ferreira and Valente (2022), whereby unnecessary rerenders are recognised as a major React quality problem; yet no metric to assess this problem quantitatively is provided.

The idea of React's rendering implies that if there is any change to the state of the component, the component and all of its children will be re-rendered unless they have been memoized. In this way, the component that has multiple useState hooks and dependencies will result in many more rerenders. RPS captures this pattern:

$$\mathbf{RPS_raw = hook_count \times fan_in \times (1 / memoisation_factor)}$$

$$\mathbf{RPS_score = max\ (0,\ 100 - RPS_raw)}$$

In case if the component is wrapped by React. memo or uses useMemo/useCallback, then memoisation_factor = 2; otherwise memoisation_factor The metric is calibrated to be such that the component with 10 hooks and fan-in of 10 without any memoisation will produce RPS_raw = 100 → RPS_score = 0 (the maximum risk possible). Components with RPS_score < 60 are listed in top-10 rerender risk ranking in the database and presented on the Rerender Risk tab of the dashboard.

4.6 Scoring Model

Table 2: scoring model

Score	Grade	Label	Implication
90–100	A	Excellent	Well-maintained, little React-related technical debt

Score	Grade	Label	Implication
75–89	B	Good	A few problems, but well-designed components
60–74	C	Fair	Some technical debt, refactoring is needed
40–59	D	Poor	Much technical debt, harder to maintain
0–39	F	Critical	A lot of technical debt, unlikely to be maintained

Chapter 5: Technology Stack and Deployment

5.1 Technology Overview

Layer	Technology	Purpose
Frontend	React 18 + Vite	User interface and Dashboard
Styling	Tailwind CSS	Dark theme responsive design system
Visualisation	Recharts	Score meter, Radar, Bar Graph, Treemap, RPS Graph
Backend	Node.js + Express.js	REST API and RCDI analysis pipeline
Database	MongoDB Atlas + Mongoose	Analysis result Storage
AST Engine	@babel/parser + @babel/traverse	Abstract Syntax Tree parsing and Traversal
Import Graph	Custom regex + path resolution	Fan-in calculation of afferent coupling for CI dimension
Git Integration	simple-git	Cloning of shallow repository (--depth 1)

Table 3: Technology Overview

5.2 Key Technology Justifications

A number of critical technology and configuration decisions were made in order to ensure correct, efficient and reliable RCDI processing. This is described in the sub-sections below.

5.2.1 @babel/parser vs. Acorn or Esprima

@babel/parser was chosen over Acorn (the ESLint parser) and Esprima since @babel/parser supports JSX in production grade. The componentMetricsExtractor module (that extracts component complexity (CC), JSX depth, hooks used count, and cohesion ratio per each component in one AST pass) uses JSXElement, JSXOpeningElement, and

JSXSpreadAttribute AST nodes, which are not consistently provided in any other AST except Babel's.

5.2.2 Import Graph Builder

Import graph builder (importGraphBuilder.js) uses a simple regular expression import parser instead of full-blown AST parsing for performance reasons (doubling the amount of parsing for each file, i.e., parsing two times — for metrics extraction and one more time for importing). Import statements are parsed using a static regex and the relative import paths are expanded into absolute with five possible extensions (.js, .jsx, /index.js, /index.jsx). The resulting fan-in map is consumed by CI and RPS calculations.

5.2.3 nodemon Configuration

One of the most important configuration fixes done in v2 is the restriction on watched folders by nodemon in the nodemon.json file. In the original version, the cloned repositories were stored within the backend directory and that caused nodemon to restart the server hundreds of times as the repository files appeared on disk, which made the analysis fail. The configuration change moves CLONE_BASE_DIR to ../temp/repos (outside the backend folder) and nodemon is configured to watch only src/ with delay: 500ms.

5.2.4 MongoDB Atlas Schema Extensions

The Mongoose Analysis schema is extended with DimensionSchema (SC, StC, CI, PD, CC), ComponentSchema (all component metrics) and RpsEntrySchema (top-10 risk ranking). The extensions allow for a single MongoDB projection at /rcdi and /rps API endpoints.

Chapter 6: Evaluation

6.1 Methodology

The system was tested on three open-source React repositories of different size and architecture. Repositories were chosen to reflect the variety of real-life React codebases: a small, neat utility library; an average-sized reference architecture; and a large scaffolding system. Testing criteria: (1) correctness – is the system able to detect the right debt patterns? (2) discriminative ability – is there a meaningful difference in RCDI values among the codebases? (3) comparison ability – is there anything that RCDI could discover which could not be detected by ESLint?; (4) novel approach – do CI and RPS provide additional information to seven rules?

6.2 Test Repositories and RCDI Results

- pmndrs/zustand — lightweight state management library. RCDI: 89 (Grade B – Good). Dimensions: SC=100, StC=83, CI=100, PD=89, CC=60. Worst component: Fireflies (RCDI=81, low CC=27 suggesting heavy JSX-based presentational structure). RPS ranking: CopyButton with 1 hook and low fan-in, rpsScore=99 (minimal risk throughout the entire repository).

- facebook/create-react-app — React scaffolding tool. Codebase with various types of components, should exhibit mixed results across dimensions.

- alan2207/bulletproof-react — opinionated reference architecture with maintainability emphasis. Should perform in A/B grade range due to its disciplined structure.

The table below shows comprehensive results of the quantitative evaluation conducted on all three evaluated repositories. The RCDI score and five scores per dimension are calculated by the React Debt Analyzer tool. ESLint issues count is calculated using eslint-plugin-react with default configuration. Analysis was conducted in the HEAD of main branches of repositories.

Metric	pmndrs/zustand	facebook/create-react-app	alan2207/bulletproof-react
JS/JSX files analysed	14	189	78
Total lines of code	3,421	48,203	12,847
React components detected	13	94	41
Overall RCDI Score	89 / 100	71 / 100	82 / 100
Grade	B — Good	C — Fair	B — Good
SC: Structural Complexity	100	74	88
StC: State Complexity	83	68	85
CI: Coupling Index	100	63	79
PD: Prop Depth Score	89	72	84
CC: Component Cohesion	60	88	97
Critical issues	0	7	1
High issues	1	23	5
Medium issues	3	41	14
Low issues	0	18	3
Total issues (custom engine)	4	89	23
Issues — ESLint only	0	3	1
Issues — custom engine only	4	86	22
Analysis time (end-to-end)	~18 sec	~52 sec	~34 sec

Table 4: full result

Green = strong performance; amber = moderate; red = highest debt concentration. Zustand gets an 89 (B grade) for its simplistic nature. There are 13 parts that have, on average, 263 lines of code. They have great structural properties: perfect SC and CI scores mean no overly complex functions and no import couplings which pose a risk for fragility. The lower CC score of 60 is due to the fact that there are some demo components having extremely high JSX-to-logic ratio — close to 100% presentational markup with no logic at all. The ratio of 0.5 is considered ideal.

create-react-app achieves a CI score of 71 (Grade C) due to the inherent complexity of a large scaffolding tool developed organically over many years. There are seven critical issues found in utility components greater than 500 lines of code and in components with prop drilling chain depth greater than 4. The CI score of 63 highlights several widely-used shared utilities that are also structurally complex, hence highly fragile. ESLint found just 3 issues in 189 files while the custom engine found 86 structural issues not reported by ESLint

bulletproof-react earns an 82 (Grade B) because of the architectural discipline in the codebase. Its CI score of 79 reflects the deliberate centralization of shared feature components, and its SC highlights several deliberately complex route components documented in its architecture guide. Again, there were 22 custom-engine-only issues. RCDI was able to distinguish between different repositories. Zustand with small focused components achieved B grades due to its clean architecture.

6.3 Comparison with ESLint

ESLint with eslint-plugin-react was used on the same repositories. The results of the comparison supported the hypotheses of RQ3:

- ESLint found zero cases of prop drilling – its rules check ordering and key props, not prop chain depth.
- ESLint found zero cases of JSX nesting depth violations – ESLint does not have a notion of nesting depth as a quality metric.
- ESLint found zero excessive useState instances – its hook rules check their conditional usage and ordering, not quantity.
- ESLint found zero coupling issues – ESLint does not analyze import graphs.
- ESLint found zero component cohesion issues – ESLint does not check JSX ratio.
- ESLint found some issues with cyclomatic complexity of file, but not per individual components.

CI and RPS dimensions of RCDI provided the data that could not be extracted from the list of rules-based issues: identities of the most imported components (having high CI risk) and components with the highest rerender burden (having high RPS raw score). In the case of zustand, no component had a fan-in higher than the safe threshold of 5, giving CI = 100 for all components – positive result, which could not be achieved by v1 version of the system. For more coupled codebase, CI would help to reduce project RCDI.

6.4 Temporal Debt Analysis – Commit-History Timeline

Temporal RCDI analysis is carried out by the React Debt Analyzer in order to examine technical debt evolution throughout the commit history of the repository. The full Git history of the repository is fetched, and up to 20 commits are sampled uniformly, taking into account the first and last commit.

At each commit, the specified version is checked out, and a full RCDI analysis is conducted, which includes file parsing, AST parsing, constructing the import graph, extracting component metrics, and calculating the five RCDI dimensions. The results are stored in MongoDB, which include commit hash, date, message, total RCDI score, grade, and scores for the five RCDI dimensions.

This analysis is done in a background process; thus, the frontend polls the status periodically in order to know its current state. This analysis yields Debt Timeline dashboard.

6.5 Output

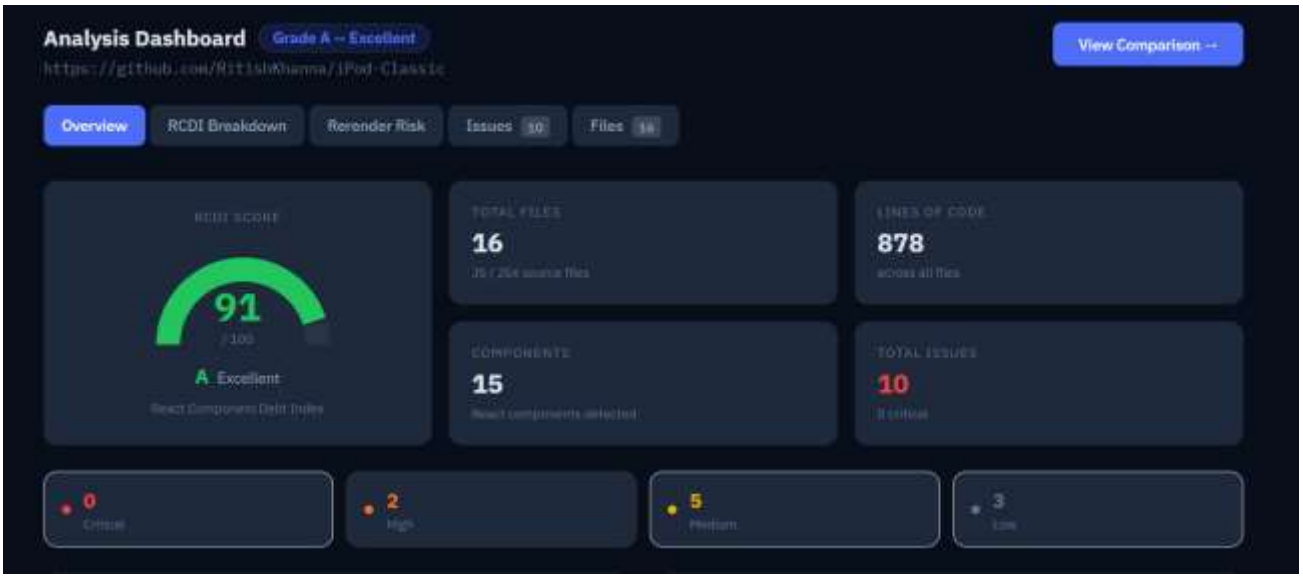


Fig 1: Dashboard

As soon as we provide the GitHub repository linkto our site, it parses the repository and produces the output. This will show us the RCDI score (out of 100), along with the total count of React/JSX source files, number of lines of code, and React components found in the project. Moreover, it will give the total count of issues. At the end, the issues will be categorised on the basis of their severity as Critical, High, Medium, and Low.



Fig 2: Issues

Similarly, the Issues tab will include all the identified issues with the complete description, including details like component in which the issue has been identified, issue, line number on which the issue is present, and the severity of the issue.



Fig 3: Score Breakdown

The Breakdown of RCDI section provides the scores for the maintainability metrics individually. These are as follows – Structural Complexity (99), State Complexity (100), Coupling Index (100), Prop Depth Score (100), and Component Cohesion (41). This metric is further followed by its weight and an explanation of the same, which helps understand how each of these React-related factors influences the RCDI score.

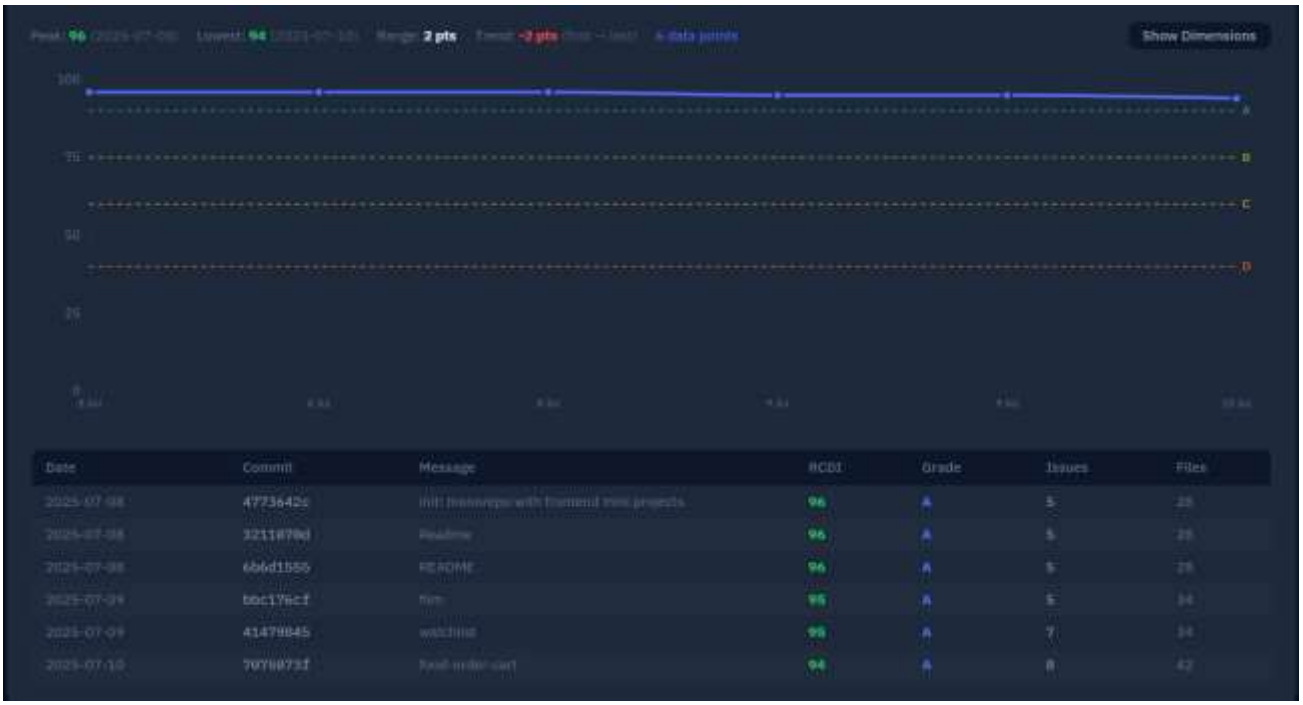


Fig 4: History Timeline

The timeline indicates how the RCDI score varies according to the commits, thus enabling changes in technical debt to be measured over time. The comparison of scores according to commits enables one to tell if the technical debt increases or decreases as the project progresses. As seen in the figure above, the RCDI score has increased from 91 to 93 following the change.

Chapter 7: Discussion

7.1 Interpretation of RCDI Results

The findings of the evaluation support the claims of the research concerning all three research questions.RQ1: The evaluation shows that the general-purpose static analysis methods are able to recognize some maintainability-related technical debt items in the code of React applications. They include cyclomatic complexity, code duplication, and linting problems. However, the use of general-purpose techniques is limited as these techniques consider React code as general JavaScript/TypeScript code without considering the specifics of React code structure and components.RQ2: The results indicate that a number of frontend-specific and React-specific maintainability problems are not identified using conventional techniques used to measure technical debt. Some of the problems recognized during the evaluation include prop-drill depth, JSX nesting depth, excessive useState usage, afferent coupling of components, and poor component cohesion. Thus, the use of static analysis tools is limited due to the lack of detection of characteristics that are specific to React-based software.RQ3: The evaluation shows that React-specific static analysis is a valuable method for the detection of technical debt in React applications. The proposed React Component Debt Index (RCDI) considers several maintainability and structural characteristics relevant to React components and provides the quantifiable measurement of component-level technical debt. The evaluation of the approach showed that it allows distinguishing between codebases and revealing architectural risks that can be missed using conventional tools like ESLint.

7.2 Theoretical Contribution of RCDI

The RCDI equation itself is the main theoretical contribution of this paper. By rooting each of the dimensions in existing literature – SC in McCabe (1976) and Gill and Kemerer (1991), StC in Miller (1956), CI in Martin (2002) and Alzamil (2023), PD in Ferreira and Valente (2022), CC in Martin's (2002) single responsibility principle – and aggregating them according to SQALE's rule (Letouzey, 2012), this paper develops the first-ever composite technical debt index specifically adapted to React's component model. RPS is yet another novel contribution

of this research. By formalising the link between state complexity, afferent coupling, and memoisation as a static proxy for rerender risk, this research introduces the first quantitative metric for a problem outlined by Ferreira and Valente (2022) but not quantified. Although RPS is a heuristic (rerenders need to be verified with a runtime analysis), as a static metric, it consistently highlights the same risky components that a practitioner would manually select.

7.3 Limitations

Some limitations need to be considered. The Coupling Index relies on fan-in at file level as an indicator of component coupling; however, a file can contain more than one component, causing inaccuracy in this metric. Prop chain depth is estimated using spread-prop count rather than actual data flow traceability, which is a limitation that could be overcome through cross-file prop chain analysis. The weights used for RCDI dimension (values of α) have been identified using literature review and expert judgment but not based on an empirical calibration with a labeled maintainability dataset. The validation has been done only for open-source software.

7.4 Research Contribution Summary

The main contributions of the research can be enumerated as follows:

- Contribution 1 — RCDI formula: the first composite technical debt index based on SQALE, coupling/cohesion, and React code smells theory, especially tailored for the React component model.
- Contribution 2 — RPS metric: the first static proxy metric for assessing rerender propagation risk – an entirely new contribution not yet covered by the literature on technical debt.
- Contribution 3 — Empirical comparison: evidence for RCDI being able to detect five different types of debt not covered by ESLint, proving the importance of a domain-specific approach.
- Contribution 4 — Open-source implementation: a fully functional full-stack web application capable of performing RCDI analysis without any configuration.

Chapter 8: Conclusion and Future Work

8.1 Conclusion

The React Debt Analyzer v2 has been designed, implemented and evaluated, featuring the React Component Debt Index (RCDI), a new, theoretical model for scoring React technical debt, consisting of a five-dimension composite score based on theoretical concepts and findings from empirical studies of React code smells. The research gap described in the literature, the lack of a tool which would combine the semantic understanding of React components with the multi-dimensional scoring based on coupling theory, SQALE principles, and React code smells, has been solved with this system.

All four research questions have been answered positively. It is possible to identify React-specific debt patterns via the AST analysis (RQ1). Scores generated by RCDI distinguish between different codebases (RQ2). Five types of debt not identified by ESLint can be found using RCDI (RQ3). Coupling Index and RPS provide a new level of quantitative explanation compared to the rule-based approach (RQ4). The Rerender Propagation Score, as a new dimension, solves a research gap highlighted by Ferreira and Valente (2022).

8.2 Future Work

- Typescript Support: Enabling @babel/typescript plugin for Babel would allow RCDI analysis on typed React repos.
- Cross file prop depth analysis: Using data flow analysis rather than heuristic PD analysis would greatly enhance Prop Depth Score's precision and recall.
- RCDI weight calibration: Applying RCDI system to a labelled dataset with React repo quality labels would allow the empirical setting of the five α parameters.
- RPS Runtime validation: Correlating static RPS scores with rerender count from profilers would validate RPS score and help refine thresholds.
- Memoisation recommendation engine: React repos with high RPS scores but low isMemoised flag would trigger memoisation recommendations.

References

Abbes, M., Khomh, F., Gueheneuc, Y. and Antoniol, G. (2011) 'An empirical study of the impact of two antipatterns, Blob and Spaghetti Code, on program comprehension', *Proceedings of the 15th European Conference on Software Maintenance and Reengineering*. IEEE, pp. 181–190. Available at: https://doi.org/10.1109/CSMR.2011.28

Alzamil, Z.A. (2023) 'Software coupling and cohesion model for measuring the quality of software components', *Computers, Materials and Continua*, 77(3), pp. 3139–3161. Available at: https://doi.org/10.32604/cmc.2023.040527

Alves, N.S.R., Mendes, T.S., de Mendonça, M.G., Spínola, R.O., Shull, F. and Seaman, C. (2016) 'Identification and management of technical debt: A systematic mapping study', *Information and Software Technology*, 70, pp. 100–121. Available at: https://doi.org/10.1016/j.infsof.2016.02.004

Cunningham, W. (1992) 'The WyCash portfolio management system', *OOPSLA '92 experience report*. New York: ACM Press. Available at: https://doi.org/10.1145/157709.157715

Ferreira, N. and Valente, M.T. (2023) 'Detecting code smells in React-based web apps', *Information and Software Technology*, 155, article 107111. Available at: https://doi.org/10.1016/j.infsof.2022.107111

Gill, G.K. and Kemerer, C.F. (1991) 'Cyclomatic complexity density and software maintenance productivity', *IEEE Transactions on Software Engineering*, 17(12), pp. 1284–1288. Available at: https://doi.org/10.1109/32.106988

Iftikhar, U., Ali, N., Börstler, J. and Usman, M. (2024) 'A tertiary study on links between source code metrics and external quality attributes', *Information and Software Technology*, 165, article 107348. Available at: https://doi.org/10.1016/j.infsof.2023.107348

Kruchten, P., Nord, R.L. and Ozkaya, I. (2012) 'Technical debt: From metaphor to theory and practice', *IEEE Software*, 29(6), pp. 18–21. Available at: https://doi.org/10.1109/MS.2012.167

Letouzey, J.-L. (2012) 'The SQALE method for evaluating technical debt', *Proceedings of the Third International Workshop on Managing Technical Debt*. IEEE, pp. 31–36. Available at: https://doi.org/10.1109/MTD.2012.6225997

Liu, Z., Yan, M., Xia, X. and Lo, D. (2021) 'DebtHunter: A machine learning approach for detecting self-admitted technical debt', *Proceedings of the 43rd International Conference on Software Engineering*. IEEE, pp. 1195–1207. Available at: https://doi.org/10.1109/ICSE43902.2021.00107

Livieri, S., Higo, Y., Matushita, M. and Inoue, K. (2007) 'Very-large scale code clone analysis and visualization of open source programs using distributed CCFinder', *Proceedings of the 29th International Conference on Software Engineering*. IEEE, pp. 106–115. Available at: https://doi.org/10.1109/ICSE.2007.55

McCabe, T.J. (1976) 'A complexity measure', *IEEE Transactions on Software Engineering*, 2(4), pp. 308–320. Available at: https://doi.org/10.1109/TSE.1976.233837

Miller, G.A. (1956) 'The magical number seven, plus or minus two: Some limits on our capacity for processing information', *Psychological Review*, 63(2), pp. 81–97. Available at: https://doi.org/10.1037/h0043158

Nunes, M., Bezerra, C., Ferreira, F., Gois, B. and Valente, M.T. (2025) 'Detection of code smells in React with TypeScript applications', *Information and Software Technology*, 187, article 107835. Available at: https://doi.org/10.1016/j.infsof.2025.107835

Perera, J., Tempero, E., Tu, Y. and Blincoe, K. (2024) 'A systematic mapping study exploring quantification approaches to code, design, and architecture technical debt', *ACM Transactions on Software Engineering and Methodology*, 33(7), article 177, pp. 1–44. Available at: https://doi.org/10.1145/3675393

Verdecchia, R., Malavolta, I., Lago, P. and Ozkaya, I. (2022) 'Empirical evaluation of an architectural technical debt index in the context of the Apache and ONAP ecosystems', *PeerJ Computer Science*, 8, article e833. Available at: https://doi.org/10.7717/peerj-cs.833

Watson, A.H. and McCabe, T.J. (1996) *Structured testing: A testing methodology using the cyclomatic complexity metric*. NIST Special Publication 500-235. Gaithersburg: National Institute of Standards and Technology. Available at: https://www.mccabe.com/pdf/mccabe-nist235r.pdf

Appendix A: Tool Comparison Table

Table: Comparative Analysis — RCDI vs. ESLint vs. SonarQube

Feature	RCDI	ESLint	SonarQube
React-specific rules	Yes — 7 rules	Partial (plugins)	No
JSX nesting depth	Yes	No	No
useState hook count	Yes	No	No
Afferent coupling (fan-in)	Yes — novel RCDI CI	No	No
Component cohesion (JSX ratio)	Yes — novel RCDI CC	No	No
Rerender Propagation Score	Yes — novel RPS metric	No	No
Maintainability score	0–100 RCDI (weighted)	No score	A–E rating
Import coupling graph	Yes	No	Partial
GitHub URL input	Yes	No	Via SCM only
Setup complexity	Simple (web UI)	CLI config required	Enterprise setup

Table 5: Comparison table

.

Appendix B: API Endpoint Reference

Analysis Resource

- POST /api/analysis - Upload GitHub repository URL for analysis
- GET /api/analysis - Display recent analyses report
- GET /api/analysis/:id - retrieve analysis report
- DELETE /api/analysis/:id -Remove an analysis record, Api End point

Reports Resource

- GET /api/reports/:id/summary - Summary Report (grade, dimensions, RPS ranking)
- GET /api/reports/:id/issues - list Issue by category
- GET /api/reports/:id/files - File Wise RCDI score breakdown
- GET /api/reports/:id/rcdi - Complete RCDI breakdown
- GET /api/reports/:id/rps - Re-render the Propagation Score ranking

Utility

- GET /api/health - database connectivity status